

SYSTEM DESIGN INTERVIEW

# Design a Payment Gateway

Stripe/PayPal-style transaction flow, idempotency, PCI compliance, and fraud detection

~ 45 minute read

© Study Guide — For personal use only

## What's Inside

---

- 01 · Problem Overview
- 02 · Clarifying Questions
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Core Design Decisions
- 06 · Data Models
- 07 · Payment Transaction Flow
- 08 · Fraud Detection & Prevention
- 09 · Settlement & Reconciliation
- 10 · Failure Modes & Recovery

[11](#) · [Design Trade-offs](#)

[12](#) · [Interview Playbook](#)

# 01

## Problem Overview

Stripe/PayPal at scale

Design a payment gateway in the spirit of **Stripe**, **Adyen**, or **PayPal**. The system must accept charges from merchants, tokenize cards, run fraud and risk checks, route to a card network, persist a double-entry ledger, and emit reliable webhooks — all while never losing or duplicating a single cent and surviving the network outages of third parties we do not control.

### Scale & Requirements

- **Volume:** ~250M transactions/day; ~2.9K TPS daily average; ~10K TPS sustained at peak hour
- **Burst:** Black Friday / Cyber Monday peaks to ~100K TPS globally for short flash windows
- **Uptime:** 99.999% (5 nines); money cannot be lost or duplicated under any failure mode
- **Latency:** gateway processing p99 < 500 ms; end-to-end (incl. card network) p99 ≤ 8–10 s
- **Compliance:** PCI DSS Level 1; never store raw PAN; tamper-evident audit trail
- **Fraud:** real-time ML scoring < 50 ms; 3DS authentication; velocity / device checks
- **Settlement:** daily batch reconciliation against card-network files; double-entry accounting

### Core Challenges

- **Idempotency:** networks time out and merchants retry — must dedup without duplicating charges
- **Double-spend:** user clicks *Pay* twice; prevent the second hitting the bank
- **Distributed transactions:** atomically update merchant balance, ledger, and fraud log across regions without 2PC
- **External dependency:** issuing banks take 1.5–8 s; need async fallback and timeout handling
- **Fraud arms race:** ML models retrain daily; balance false positives (lost revenue) vs. false negatives (chargebacks)
- **Compliance:** PCI DSS, PSD2, SOX, regional regs; a single breach is a \$5–25M event

#### WARNING

##### The Headline Tension

Payments has two competing obsessions: **correctness** (no double-charge, no lost money, audit on demand) and **availability** (5 nines despite a card network you do not own). Almost every architectural choice picks correctness for the money path and availability for everything else.

## 02

## Clarifying Questions

Scope refinement

Spend the first three minutes pinning scope. Payments has more branching than almost any other interview question — clarify what is in and out before drawing a single box.

| Question        | Options                      | Assumption                                      |
|-----------------|------------------------------|-------------------------------------------------|
| Payment types?  | Cards, ACH, wallets, crypto  | Credit/debit cards (extend later)               |
| Multi-currency? | Single (USD) or global       | Multi-currency (100+); FX at capture            |
| Regions?        | US-only or global            | Global; regional regs differ (PSD2, etc.)       |
| Merchants?      | Small / large / platforms    | Mix; platforms (e.g. Shopify) need sub-accounts |
| PCI Level?      | 1, 2, or 3                   | Level 1 (strictest; > 6M txn/yr)                |
| Retry logic?    | Auto-retry? Cached fallback? | 3 retries w/ backoff; soft decline ≠ retry      |
| Refunds?        | Full / partial? Chargebacks? | Full + partial; full chargeback flow            |
| Latency budget? | Sync vs async charge?        | Sync auth; async capture & webhook              |
| Tokenization?   | In-house vault or PSP?       | In-house HSM-backed vault (PCI scope)           |

### TIP

#### What the Interviewer Is Listening For

The strongest signal in the first five minutes is whether you *separate* the things you can control (your code, your DBs, your fraud model) from the things you cannot (card networks, issuing banks). That separation drives every SLO, retry, and fallback decision later in the design.

# 03

## Capacity Estimation

Math for scale

### Traffic Estimation

---

- Daily transactions: **~250M baseline** → ~10K TPS sustained at peak hour (Stripe-scale), ~2.9K TPS daily average
- Black Friday / Cyber Monday peak: bursts to **~100K TPS** globally for short windows; design headroom to 2× peak (~200K TPS) so we do not shed load during the 30-second flash window when every checkout in the world fires at once
- Read:Write profile: roughly **4:1** — every charge generates lookups (idempotency check, fraud features, merchant config, ledger reads for refunds/chargebacks) on top of the write
- Webhook fan-out: 1 charge → ~3 outbound events (authorized, captured, settled) → **~30K webhook deliveries/sec** at peak; size Kafka and the retry tier for that, not for the raw charge rate

### Storage Estimation

---

- Per payment record: ~2 KB hot (UUIDs, amounts, status, auth\_code, fraud\_score, timestamps) + ~3 KB extended (addresses, device fingerprint, 3DS metadata) → budget **~5 KB/record** including indexes
- Daily payments storage:  $250M \times 2 \text{ KB} = \text{~500 GB/day hot}$ ; ~1.25 TB/day if extended fields and per-row indexes are co-located
- Ledger entries: 3–4 per charge → ~1B rows/day  $\times$  ~300 B = **~300 GB/day**
- Retention: **7 years minimum** (PCI DSS, SOX, most national tax regimes; some EU regimes require 10)
- Payments hot (1 year online): **~180 TB**
- Payments warm/archive (years 2–7): ~1.1 PB in object storage / Glacier
- Ledger hot (1 year): **~110 TB**; archive (years 2–7): ~660 TB
- Idempotency cache (Redis):  $250M \text{ keys/day} \times 24 \text{ h TTL} \times \sim 10 \text{ KB cached response} = \text{~2.5 TB}$ ; cluster sized at 4 TB to leave headroom

### Bandwidth Estimation

---

- Inbound API:  $10K \text{ TPS} \times \sim 2 \text{ KB request body} \approx \text{20 MB/s sustained}$ , 200 MB/s at Black Friday peak
- Card-network egress:  $10K \text{ TPS} \times \sim 1 \text{ KB ISO 8583 frame} \approx \text{10 MB/s}$ ; replicated across regional acquirers
- Webhook egress:  $30K \text{ events/sec} \times \sim 1 \text{ KB} \approx \text{30 MB/s sustained}$ , ~300 MB/s peak
- Kafka inter-broker: ~3× replication factor on a ~50 MB/s topic mix → **~150 MB/s** on the cluster backbone

## Latency Targets (separated budgets)

---

- **Gateway processing** (everything we own: API gateway → fraud → risk → ledger write, *excluding* card-network round-trip): p99 < 500 ms, p50 < 150 ms
- **Card-network + issuing-bank round-trip** (out of our control): typically 1.5–3 s p50, 5–8 s p99 over ISO 8583 / network APIs
- **End-to-end authorization** (gateway + network + bank): p99 ≤ 8–10 s, p50 ≈ 2 s. SLOs are defined separately so a slow issuer cannot blow our processing SLO
- **Fraud scoring**: p99 < 50 ms (in-process model server)
- **Primary DB write** (ledger insert + idempotency upsert): p99 < 20 ms

### KEY

#### Key Numbers

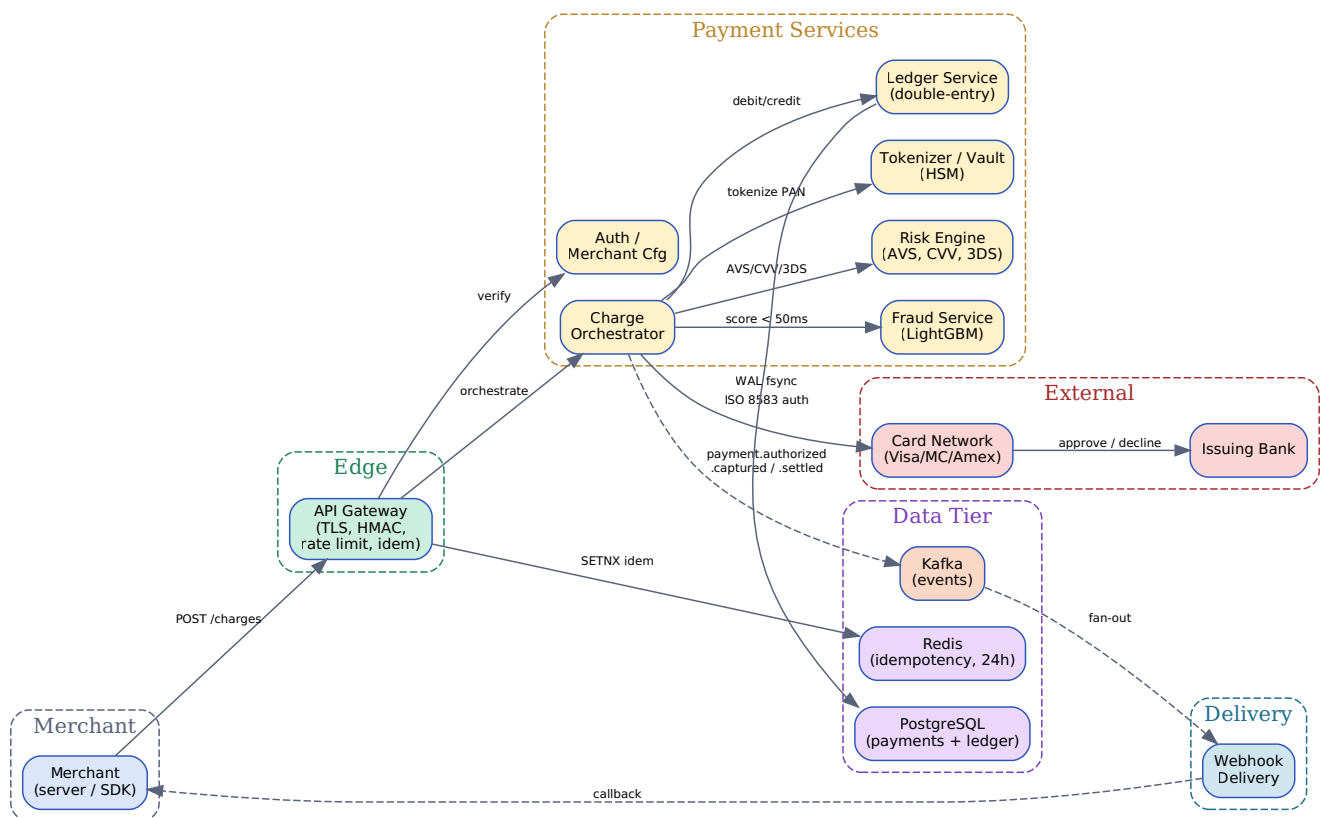
Sustained TPS: **~10K** · Peak TPS (Black Friday): **~100K** · Hot payments storage (1 yr): **~180 TB** · Hot ledger storage (1 yr): **~110 TB** · Idempotency cache: **~2.5 TB** Redis (24 h TTL) · Retention: **7 years** (PCI / SOX)

# 04

## High-Level Architecture

System components and data flow

Merchants submit charges via REST to the **API Gateway** (TLS, idempotency, rate-limiting). The **Payment Processor** orchestrates fraud detection, risk checks, tokenization, and card-network authorization. Data is persisted to **PostgreSQL** (payments + double-entry ledger), **Redis** (idempotency cache, 24h TTL), and **Kafka** (async events feeding webhooks, analytics, settlement). Separate **Settlement** and **Refund** services handle batch reconciliation and reversals.



High-level architecture: merchant → API gateway → payment processor → {auth, fraud, risk, vault, charge, ledger} → card network → issuing bank. Ledger writes hit PostgreSQL synchronously; downstream events flow through Kafka to webhook delivery.

### Tier 1: API Gateway

- **Clients:** REST API, mobile SDKs, server-to-server webhooks
- **Rate limiting:** token bucket per merchant key (per-second + per-day)
- **Auth:** TLS 1.3 + HMAC-SHA256 signature verification on every request
- **Routing:** Route53 latency-based; multi-region active-active
- **Idempotency:** first stop for the Idempotency-Key header (see §05)

## Tier 2: Payment Services

---

- **Charge Orchestrator:** drives the saga; calls fraud, risk, vault, network in order
- **Fraud:** LightGBM model server; real-time scoring p99 < 50 ms; score 0–100
- **Risk:** AVS, CVV, 3DS, velocity checks; deterministic rule engine
- **Tokenizer / Vault:** HSM-backed; raw PAN never leaves this tier
- **Ledger:** double-entry, append-only; 3–4 entries per charge
- **Reconciliation:** daily batch matching against card-network files; discrepancy alerts

## Tier 3: Data Layer

---

- **PostgreSQL:** ACID; synchronous WAL; multi-region primary/replica with controlled failover
- **Redis:** idempotency cache, 24 h TTL; cluster mode + AOF for durability
- **Kafka:** async events (`payment.authorized`, `.captured`, `.refunded`, `.settled`); consumers: webhooks, analytics, settlement
- **Object store:** S3 / Glacier for warm + cold ledger and audit archives (years 2–7)

## Tier 4: External & Compliance

---

- **Card Networks:** Visa, Mastercard, Amex; ISO 8583 + REST APIs
- **Acquirers:** banks processing on the merchant's behalf
- **Issuing Banks:** consumer banks; approve/decline based on funds + fraud rules
- **PCI DSS Vault:** HSM tokenizes raw cards at the edge; PAN never enters app code or primary DB
- **Audit logs:** immutable append-only; every transaction, every action; PCI requirement



# 05

## Core Design Decisions

Idempotency, at-least-once, double-spend

### Decision 1: Idempotency Keys

- **Problem:** merchant clicks *Pay*; network times out; merchant retries — duplicate charge?
- **Solution:** client supplies Idempotency-Key (UUID); server stores key → response in Redis for 24 h
- **Mechanism:** first request processes and stores; retry returns the cached response immediately
- **Guarantee:** exactly-once semantics at the API layer; no double charges
- **Durability:** Redis row mirrored to a durable `idempotency_keys` table so cache loss does not collapse the guarantee

### Idempotency Key Handling — Reference Implementation

Every `POST /charges` requires the merchant to supply an Idempotency-Key header (a client-generated UUID). The gateway treats `(merchant_id, idempotency_key)` as the deduplication tuple. Replays are answered from cache; replays with a different body for the same key are rejected with 409.

```
import hashlib, json, redis

r = redis.Redis()
TTL_SECONDS = 24 * 60 * 60 # 24h, matches Redis key retention

def handle_charge(merchant_id, idempotency_key, request_body):
    cache_key = f"idem:{merchant_id}:{idempotency_key}"
    body_hash = hashlib.sha256(
        json.dumps(request_body, sort_keys=True).encode()
    ).hexdigest()

    # SETNX = atomic 'claim this key if nobody else has'.
    # Value initially holds the request fingerprint + a PENDING marker.
    claimed = r.set(
        cache_key,
        json.dumps({"status": "PENDING", "body_hash": body_hash}),
        nx=True, ex=TTL_SECONDS,
    )

    if not claimed:
        existing = json.loads(r.get(cache_key))
        if existing["body_hash"] != body_hash:
            # Same key, different payload -> client bug or replay attack.
            return 409, {"error": "idempotency_key_reuse_with_different_payload"}
        if existing["status"] == "PENDING":
            # Original request still in-flight -> tell client to retry.
```

```

        return 409, {"error": "request_in_progress"}
    # Completed: replay the cached response verbatim.
    return existing["status_code"], existing["response"]

# First time we have seen this key -- actually run the charge.
try:
    status_code, response = process_payment(merchant_id, request_body)
except Exception:
    # Release the slot so the merchant's retry can proceed.
    r.delete(cache_key)
    raise

r.set(
    cache_key,
    json.dumps({
        "status": "DONE",
        "body_hash": body_hash,
        "status_code": status_code,
        "response": response,
    }),
    ex=TTL_SECONDS,
)
return status_code, response

```

## INSIGHT

### Why This Shape

SET ... NX EX is the atomic claim primitive (single round-trip; survives concurrent retries from the same merchant). The **PENDING** marker lets us distinguish 'in-flight' from 'already finished' so a fast retry returns 409 instead of double-charging. The body fingerprint protects against keys being reused by a buggy client for a different amount/currency/customer. The Redis row is mirrored to the durable `idempotency_keys` table (see §06) so cache loss does not collapse the guarantee — on Redis miss, the API falls back to the DB row.

## Decision 2: At-Least-Once Webhooks (Kafka)

- **Problem:** payment completes; webhook to merchant fails; merchant never learns
- **Solution:** publish event to Kafka; retry queue with exponential backoff (1 s, 10 s, 100 s, 1000 s); max 5 retries then DLQ
- **Guarantee:** at-least-once delivery; merchant *may* see duplicates and must dedup by `event_id`
- **Why not exactly-once?** exactly-once is unimplementable across an arbitrary merchant network — pushing dedup to the merchant is industry standard

### Decision 3: Double-Spend Prevention

---

- **Race condition:** two retries arrive simultaneously and both pass the Redis check before SETNX completes (rare but possible across clusters)
- **Defence in depth:** Redis SETNX is the fast path; PostgreSQL UNIQUE (merchant\_id, idempotency\_key) is the durable backstop
- **Advisory locks:** pg\_advisory\_xact\_lock(hashtext(idempotency\_key)) serializes any sliver that escapes the cache
- **Heuristic backstop:** flag charges < 1 s apart with overlapping amounts as suspicious for manual review

# 06

## Data Models

Schema and storage

### Payments Table (PostgreSQL)

```
CREATE TABLE payments (  
  id                UUID PRIMARY KEY,  
  merchant_id       UUID NOT NULL,  
  idempotency_key   UUID,  
  amount_cents      BIGINT NOT NULL,  
  currency          CHAR(3),           -- USD, EUR, GBP, ...  
  status            payment_status,    -- pending|authorized|captured|declined|refunded  
  card_token        VARCHAR(255),      -- tokenized; never raw PAN  
  fraud_score       DECIMAL(5,2),      -- 0-100; >80 = manual review  
  auth_code         VARCHAR(10),       -- from card network  
  created_at        TIMESTAMPTZ DEFAULT NOW(),  
  captured_at       TIMESTAMPTZ,  
  settled_at        TIMESTAMPTZ,  
  UNIQUE (merchant_id, idempotency_key),  
  INDEX (merchant_id, created_at),  
  INDEX (status, created_at)  
);
```

### Ledger Entries (Double-Entry Accounting)

```
CREATE TABLE ledger_entries (  
  id                UUID PRIMARY KEY,  
  transaction_id    UUID NOT NULL,      -- groups 3-4 entries  
  account_id        UUID NOT NULL,  
  debit_cents       BIGINT NOT NULL DEFAULT 0,  
  credit_cents      BIGINT NOT NULL DEFAULT 0,  
  currency          CHAR(3),  
  description       VARCHAR(255),       -- 'Charge', 'Fee', 'Reversal', ...  
  created_at        TIMESTAMPTZ DEFAULT NOW(),  
  settled_at        TIMESTAMPTZ,  
  INDEX (account_id, created_at),  
  INDEX (transaction_id)  
);  
  
-- Invariant (verified nightly): SUM(debit_cents) = SUM(credit_cents)  
-- across the whole table, and per-transaction inside one transaction_id.  
-- The table is APPEND-ONLY: corrections are new rows, not UPDATES.
```

## Idempotency Keys (durable mirror)

```
-- Redis (fast path):
Key:   'idem:{merchant_id}:{idempotency_key}'
Value: {status, body_hash, status_code, response}
TTL:   86,400 seconds (24h)

-- PostgreSQL (durable backstop):
CREATE TABLE idempotency_keys (
  key          UUID,
  merchant_id  UUID,
  body_hash    CHAR(64),
  response_body JSONB,
  status_code  SMALLINT,
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  expires_at   TIMESTAMPTZ,
  PRIMARY KEY (merchant_id, key)
);
```

### TIP

#### Why Append-Only Ledger

Auditable by construction ( $\text{SUM}(\text{debits}) = \text{SUM}(\text{credits})$  is a one-line invariant). No destructive updates means no corruption from a buggy migration. Easy to replay forward from any timestamp. Trade-off: storage growth (every correction is a new row) and that 'current balance' requires a materialized view.

# 07

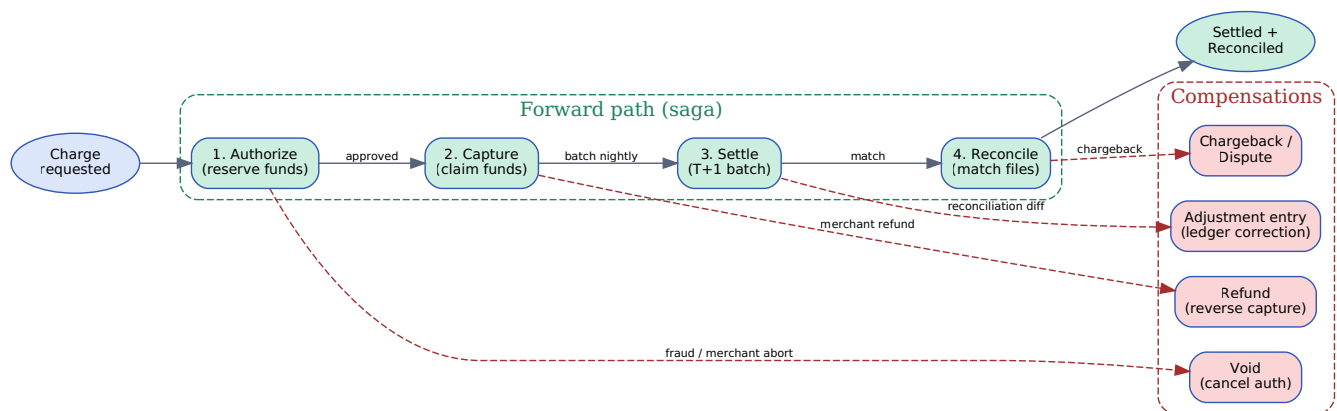
## Payment Transaction Flow

Complete lifecycle as a saga

A charge is a **saga**: a sequence of local transactions, each with a compensating action. We never use 2PC across services — it blocks on the coordinator, does not survive cross-region partitions, and is incompatible with five-nines targets that depend on third-party banks.

### Step-by-Step (happy path)

1. Merchant POST /charges with Idempotency-Key header
2. API Gateway: TLS, HMAC verify, rate-limit, dedup via Redis SETNX (returns cached response on retry)
3. Charge Orchestrator: insert payments row in pending state (single SQL txn)
4. Fraud Service: ML scoring (LightGBM, p99 < 50 ms); if score > threshold, decline or trigger 3DS
5. Risk Engine: AVS, CVV, 3DS, velocity checks (deterministic rules)
6. Tokenizer/Vault: turn raw PAN (if present) into a vault token; PAN never reaches app code or DB
7. Card Network: ISO 8583 / REST authorization; await issuing-bank approval (1.5–3 s p50)
8. Ledger Service: write 3–4 double-entry rows in one PG transaction with synchronous WAL
9. Status: flip payments.status from pending → authorized
10. Publish payment.authorized to Kafka (async fan-out to webhook, analytics, settlement)
11. Cache the final response under the idempotency key (PENDING → DONE)
12. Return 200 OK to the merchant



Charge as a saga: each forward step has an explicit compensating action. Authorize and Capture are the synchronous money-path; Settle and Reconcile are the daily T+1 batch tail.

## Compensations (the unhappy paths)

| Forward action | Compensation         | When it fires                                                    |
|----------------|----------------------|------------------------------------------------------------------|
| Authorize      | Void                 | Fraud verdict flips post-auth, or merchant aborts before capture |
| Capture        | Refund               | Merchant-initiated refund; reverses ledger entries with new rows |
| Settle         | Adjustment entry     | Reconciliation finds a discrepancy vs the network file           |
| Reconcile      | Chargeback / dispute | Cardholder disputes; funds reverse, evidence package built       |

### WARNING

#### Every Step Must Be Idempotent

Sagas only work if every forward action and every compensation can be replayed without side effects. Authorize uses the merchant's Idempotency-Key; capture and refund use the `payment_id` as the dedup key; ledger writes are guarded by `UNIQUE (transaction_id, account_id, side)`.

# 08

## Fraud Detection & Prevention

ML + rules + 3DS

### Real-Time ML Model

- **Algorithm:** LightGBM (gradient boosted trees); retrained daily on the last 30 days
- **Features:** merchant (MCC, chargeback ratio), card (BIN age, country), user (device, IP, geo, velocity), transaction (amount, currency, MCC)
- **Output:** `fraud_score` 0–100; merchant sets thresholds (e.g. 70 = decline, 85 = manual review)
- **Latency:** p99 < 50 ms; served via Seldon / KServe / in-process
- **Monitoring:** precision, recall, AUC-ROC daily; rebalance for class imbalance (fraud ~0.5%)
- **Model server:** shadow + A/B traffic; kill-switch back to last-good model

### Deterministic Rules + 3DS

- **Velocity:** same card 5× in 5 min → decline; 3× different merchants in 1 min → review
- **Geo anomalies:** card issued US, txn from RU 1 h ago, now UK → impossible-travel risk
- **3DS:** cardholder SMS OTP or biometric; 30–120 s latency; *liability shift* to issuer if authenticated
- **Device fingerprinting:** client-side JS collects user-agent, screen, timezone, IP; detects stolen-creds fingerprints
- **Allow / deny lists:** per-merchant overrides for known good/bad customers

#### INSIGHT

##### Why ML + Rules Together

Pure rules are explainable but bleed against adaptive fraud rings. Pure ML catches novel patterns but is a black box that regulators and risk teams hate. The combination — ML for the score, rules for hard guardrails (velocity, BIN bans, AML lists) — is the industry standard.



# 09

## Settlement & Reconciliation

Daily accounting cycle

Authorization promises money. **Settlement** moves it. Card networks send daily settlement files (typically T+1 to T+3); we match each line to a payment's row and to its ledger entries. Discrepancies are the highest-priority alert in the system — they mean either the network or our ledger is wrong, and one of them is going to cost real money.

### Daily Batch Job (2 AM UTC)

1. Pull settlement file from each card network (Visa, Mastercard, Amex)
2. Parse rows into a staging table: (network\_txn\_id, payment\_id, amount, fee, status)
3. Match against payments by payment\_id + amount\_cents within tolerance
4. Matched rows: insert ledger **settlement entries** (debit clearing account, credit merchant balance, debit fee)
5. Mark payments.settled\_at; emit payment.settled to Kafka
6. Unmatched rows: write to reconciliation\_discrepancies; page on-call
7. Nightly invariant check: SUM(debits) = SUM(credits) per account and globally

### Refund Path

- Merchant calls POST /refunds with payment\_id + (optional) amount
- Validate: payment is captured, refund <= captured amount, idempotency key supplied
- Reverse authorization on the card network; await ack
- Append ledger entries: debit merchant balance, credit cardholder clearing account
- Emit payment.refunded; cardholder sees funds in 3–5 business days

### Discrepancy Categories

| Category                 | Likely cause                     | Action                                           |
|--------------------------|----------------------------------|--------------------------------------------------|
| Network has, we don't    | Lost write; double-charge risk   | Manual review; insert payment + ledger or refund |
| We have, network doesn't | Auth never settled; expired hold | Mark expired; reverse ledger; notify merchant    |
| Amount mismatch          | FX rounding; partial capture     | Adjustment entry; root-cause within 24 h         |
| Fee mismatch             | Tariff change or pricing bug     | Reprice; adjustment entry; finance review        |

## KEY

### The Invariant

$\text{SUM}(\text{debit\_cents}) = \text{SUM}(\text{credit\_cents})$ . Verified per transaction, per account, per day, and globally. If this number is ever non-zero, every senior engineer in the company gets paged. The ledger is the source of truth — not the database, not the network file, the ledger.

# 10

## Failure Modes & Recovery

What breaks and how we survive it

| Failure              | Cause                                     | Impact                                          | Mitigation                                                                  |
|----------------------|-------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------|
| Bank Timeout         | Issuing bank API > 5 s                    | Auth hangs; client retries; potential duplicate | Hard 5 s timeout; soft decline; idempotency prevents duplicate              |
| Network Partition    | ISP / CDN routing down                    | Merchants can't reach API                       | Multi-region active-active; Route53 failover < 30 s                         |
| Duplicate Charge     | Retry same idempotency key                | Same charge twice (would be)                    | Redis SETNX + DB UNIQUE; cached response replay                             |
| Fraud False Positive | ML model too strict                       | Good customer declined; lost revenue            | A/B threshold tests; FP rate dashboard; merchant-tunable thresholds         |
| PCI DSS Breach       | DB hacked; card data exposed              | Regulatory fine \$5–25M; brand damage           | Tokenize at edge; PAN never in app code; HSM vault; PCI audit               |
| Card Network Outage  | Visa/MC APIs down (rare)                  | Cannot authorize any txns                       | Cached pre-approved BIN list; local fallback rules; route to backup network |
| Redis Eviction       | Memory full; old keys evicted prematurely | Idem key expired; duplicate charge possible     | Size cluster at 4 TB (1.6× working set); fall back to durable PG row        |
| Kafka Down           | Broker cluster unavailable                | Webhooks stop; settlement delayed               | Local outbox table; replay on recovery; PG remains source of truth          |
| Ledger Drift         | Reconciliation mismatch vs network        | Money in flight; audit risk                     | Page on-call; freeze affected merchant; manual reconcile within 24 h        |

### Disaster Recovery

- **RTO:** < 60 s for a regional API outage (Route53 failover); < 5 min for a primary DB failover
- **RPO:** 0 for ledger writes (synchronous WAL replication); < 1 min for analytics
- **Backups:** continuous WAL archiving + nightly snapshot; 30-day retention online, 7 yr in Glacier
- **Failover drills:** monthly chaos exercises; quarterly full-region failover; tested pgmgr scripts

## WARNING

### Graceful Degradation Order

Under load shedding, drop in this order: (1) analytics enrichment, (2) webhook delivery (deliver later), (3) recommendations / nice-to-haves, (4) *never* the money path. Authorization, ledger write, and idempotency are the last things to break.

# 11

## Design Trade-offs

Decisions and rationale

| Decision                      | Choice                                                                                                | Trade-off                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Distributed transaction model | Saga (event-driven compensation across processor → fraud → card network → ledger)                     | Survives partial outages, each step idempotent, scales horizontally; trade-off is transient inconsistency windows ("authorized but not yet captured") and the requirement to design every compensation explicitly. Alternative 2PC gives atomicity across services but blocks on the coordinator and is incompatible with 5-nines targets that depend on external third-party banks.                                                          |
| Settlement                    | Async, daily batch (T+1) reconciliation against card-network files                                    | Matches how Visa/Mastercard actually settle, batches I/O cheaply, allows nightly invariant checks (sum debits = sum credits); trade-off is that money is "in flight" for 24–72 h and discrepancies are not detected in real time. Alternative real-time settlement requires direct network membership and offers limited benefit for typical PSP-style integrations.                                                                          |
| Card-network integration      | Integrate with PSPs / acquirers (Stripe-style) rather than direct network membership                  | Faster to launch, no PCI Level 1 acquirer compliance from day one, fewer bank contracts; trade-off is per-transaction fees, dependence on the PSP's uptime, and the PSP's own rate limits. Direct membership is cheaper at very high volume but takes 12–24 months and a Level-1 audit.                                                                                                                                                       |
| Idempotency mechanism         | Client-supplied Idempotency-Key (UUID), dedup via Redis SETNX with TTL + durable mirror in PostgreSQL | Constant-time check, exact replay of original response, easy for merchants to reason about; trade-off is that merchants must generate stable keys (a real implementation problem) and Redis must be sized to never evict before TTL. Alternative request-content hashing dedups even when the merchant forgets the header but cannot tell a "legitimate retry of an identical charge" from a "two real charges that happen to look the same". |
| Ledger model                  | Append-only, immutable, double-entry                                                                  | Auditable by construction ( $\text{SUM}(\text{debits}) = \text{SUM}(\text{credits})$ is a 1-line invariant), no destructive updates means no corruption from a buggy migration, easy to replay; trade-off is storage growth (every correction is a new row) and that "current balance" requires a materialized view. Alternative mutable ledger is cheaper to query but loses the audit guarantees regulators require.                        |
| Webhook delivery              | At-least-once with exponential backoff (1s, 10s, 100s, 1000s, ...);                                   | Delivery survives merchant outages up to ~hours, no central coordinator needed; trade-off is that merchants will see duplicates and must keep their own dedup table. Alternative                                                                                                                                                                                                                                                              |

| Decision                      | Choice                                                                        | Trade-off                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                               | merchants must dedup by event_id                                              | exactly-once delivery is unimplementable across an arbitrary merchant network — pushing dedup to the merchant is the industry standard.                                                                                                                                                                                                                                   |
| PCI scope                     | Tokenize at the edge; raw PAN never enters application code or the primary DB | Shrinks the PCI DSS audit boundary from "everything" to just the HSM/vault tier, which materially reduces audit cost and breach surface; trade-off is one extra hop on every charge and a hard dependency on the vault's uptime. Alternative storing raw PAN simplifies some flows but pulls the entire stack into PCI scope and turns every breach into a \$5–25M event. |
| Fraud scoring                 | Real-time inline ML (LightGBM, p99 < 50 ms) + offline daily retraining        | Catches novel fraud patterns within 24 h, latency stays inside the auth budget; trade-off is concept drift between retrains and the operational cost of a model server in the auth path. Alternative pure rules engine is cheaper and explainable but bleeds against adaptive fraud rings.                                                                                |
| Consistency for ledger writes | Strong (single-writer Postgres primary, synchronous WAL, multi-AZ replicas)   | Money never disappears or duplicates; trade-off is that the write path is bounded by primary throughput and a regional outage requires a controlled failover (RTO seconds, RPO 0). Alternative multi-master / eventually-consistent ledger is faster to write but cannot guarantee the double-entry invariant under partition.                                            |

## KEY

### Headline Tension

Payments has two competing obsessions: **correctness** (no double-charge, no lost money, audit on demand) and **availability** (5 nines despite a card network you do not own). Almost every choice in the table picks *correctness* for the money path (strong consistency, append-only ledger, idempotency, tokenization) and *availability* for everything else (sagas, async webhooks, batch settlement, regional active-active).

# 12

## Interview Playbook

45-minute presentation

This guide distills a 45-minute payment-gateway interview into a structured narrative. Use it to practice clarifying scope, framing the correctness/availability tension, and defending trade-offs with specific numbers.

### Timeline: 5 + 3 + 10 + 20 + 7 minutes

---

1. **Opening (5 min):** clarifying questions (payment types? regions? PCI level?) → state assumptions
2. **Problem framing (3 min):** 250M txn/day → ~10K TPS sustained, ~100K peak; card network is 1.5–8 s; everything we own must be sub-second
3. **Architecture (10 min):** show Diagram 1; explain 4 tiers; solid arrows = sync money path, dashed = async events
4. **Deep dives (20 min):** pick 2–3: idempotency keys (with code), saga + compensations (Diagram 2), fraud ML, ledger invariant, reconciliation
5. **Wrap-up (7 min):** trade-offs (at-least-once vs exactly-once; batch vs real-time settlement); compliance complexity; closing on the headline tension

### Key Numbers to Memorize

---

- **Volume:** 250M txn/day; ~10K TPS sustained; ~100K TPS Black Friday peak
- **Latency:** gateway p99 < 500 ms; fraud < 50 ms; DB write < 20 ms; end-to-end p99 8–10 s
- **Card-network round-trip:** 1.5–3 s p50, 5–8 s p99 (out of our control)
- **Uptime:** 99.999% ≈ **5 min 16 sec/year** (≈ 315 sec; ≈ 26 sec/month). Six nines would be ~31 sec/year — out of reach for any system that depends on third-party card networks
- **Idempotency cache:** ~2.5 TB Redis (24 h TTL); cluster sized at 4 TB
- **Storage:** ~180 TB hot payments (1 yr); ~110 TB hot ledger; 7-year retention (PCI/SOX)
- **Ledger:** 3–4 entries/charge; ~1B rows/day; invariant  $\text{SUM}(\text{debits}) = \text{SUM}(\text{credits})$
- **Settlement:** daily batch; T+1 to T+3 latency

### Common Probe Questions & Answers

---

- **Q: How do you prevent double-charges?** A: Client-supplied Idempotency-Key + Redis SETNX with PENDING/DONE marker + body-hash 409; durable backstop in PG UNIQUE.
- **Q: Card-network timeout — what do you do?** A: Hard 5 s timeout, soft decline (do *not* retry on the customer's behalf), idempotency makes the merchant's retry safe.
- **Q: Why double-entry accounting?** A:  $\text{SUM}(\text{debits}) = \text{SUM}(\text{credits})$  is a one-line audit invariant; append-only means no destructive corruption; mandatory for SOX.

- **Q: How does 3DS work?** A: Cardholder authenticates (OTP / biometric); on success, liability shifts from merchant to issuer.
- **Q: Multi-region or single?** A: Active-active across regions for the API and read path; primary-replica with controlled failover for the ledger (correctness over availability for money).
- **Q: 2PC or saga?** A: Saga every time. 2PC blocks on the coordinator and cannot survive a card-network partition; we'd miss our 5-nines target on the first bad day.
- **Q: What about exactly-once webhooks?** A: Unimplementable across an arbitrary merchant network — we deliver at-least-once and the merchant dedups by `event_id`.

## KEY

### Memorize These Numbers

10K TPS sustained · 100K TPS peak · p99 gateway 500 ms · p99 end-to-end 8–10 s · 99.999% ≈ 5 min 16 sec/year · 2.5 TB idem cache · 180 TB hot payments · 3–4 ledger entries/charge · T+1 settlement

## INSIGHT

### Closing Line

*"Payment systems demand obsessive correctness and resilience. Idempotency is the single most important contract in the API. Test failure modes relentlessly: mock timeouts, simulate database failures, verify the ledger invariant nightly. Everything else is in service of: never lose a cent, never charge twice, prove it on demand."*